Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Department of CSE

B.Tech. CSE Trimester -5

Roll Number:2520030609
Name:V.Siri Chandana

Generative AI and AI Platforms

25CS2207E

WEEK 1 – GENERATIVE AI

First LLM API Call Using OpenAI

1. Aim

To connect a Python application to a Large Language Model (LLM) using the OpenAI API and generate an AI response by providing a user prompt.

2. Problem Statement

To build a Python application that connects to an LLM using an API key, sends a question and displays the generated response.

3. Learning Objectives

1. Understand the role of an API key in connecting to an LLM.
2. Install and use the OpenAI Python SDK.
3. Store API keys securely using a .env file.
4. Send prompts to an OpenAI model.
5. Generate and display AI responses.
6. Understand common API errors and their solutions.

4. Software Requirements

1. Python (version 3.14 or above, as specified in the lab guide).
2. Visual Studio Code / PyCharm.
3. Existing Python virtual environment (.venv).
4. OpenAI API key.
5. Internet connection.
6. Required Python libraries:
    - openai
    - python-dotenv

5. Theory

Generative AI is a type of Artificial Intelligence that can generate new content such as text, images and code.

A Large Language Model (LLM) is an AI model that understands user prompts and generates meaningful responses.

An API (Application Programming Interface) allows two applications to communicate with each other.

An API key is a unique secret key used to authenticate an application when accessing an API.

The .env file is used to store API keys securely instead of directly writing them in Python code.

6. Procedure

1. Opened the existing Week 1 project folder in Visual Studio Code.
2. Opened the integrated terminal.
3. Activated the existing Python virtual environment (.venv).
4. Created a .env file in the project folder.
5. Stored the OpenAI API key in the .env file using the variable OPENAI_API_KEY.
6. Created a .gitignore file to prevent sensitive files from being uploaded to Git.
7. Created a main.py file.
8. Imported the required libraries such as os, dotenv and openai.
9. Used load_dotenv() to load the API key from the .env file.
10. Used os.getenv() to read the API key securely.
11. Created an OpenAI client using the API key.
12. Used client.responses.create() to send a prompt to the selected AI model.
13. Displayed the generated response using response.output_text.
14. Created a requirements.txt file containing the required Python packages.
15. Installed the dependencies using python -m pip install -r requirements.txt.
16. Executed the program using python main.py.

7.Code

Create main.py

Create a file named main.py in the same folder as .env. Copy the complete code below:

```
import os
from dotenv import load_dotenv
from openai import OpenAI

# Load variables from the .env file
load_dotenv()

# Read the API key securely from the environment
api_key = os.getenv("OPENAI_API_KEY")

if not api_key:
    raise ValueError(
        "OPENAI_API_KEY was not found. Check that .env exists "
        "and contains OPENAI_API_KEY=your_key."
    )

# Create the OpenAI client
client = OpenAI(api_key=api_key)

# Send a prompt to the model
response = client.responses.create(
    model="gpt-5.5",
    input="What is Artificial Intelligence? Explain in simple words."
)

# Display the generated answer
print("\nLLM Response:")
print(response.output_text)
```

This example uses the OpenAI Responses API and the model ID gpt-5.5 as an illustrative model choice. Model availability and API access depend on your account. If the model is unavailable to your project, select a model that is listed as available to your API account and replace the model value.

Note: Code for Gemini API Key

main.py

```
from dotenv import load_dotenv
from google import genai
import os

# Step 1: Load API key from .env
load_dotenv()

# Step 2: Read Gemini API key
api_key = os.getenv("GEMINI_API_KEY")

if not api_key:
    raise ValueError("Gemini API key not found!")

# Step 3: Initialize Gemini client
client = genai.Client(api_key=api_key)

# Step 4: Make first LLM call
response = client.models.generate_content(
    model="gemini-3.8-flash",
    contents="What is Artificial Intelligence? Explain in simple words."
)

# Step 5: Display response
print("\nLLM Response:")
print(response.text)
```

8. Observation

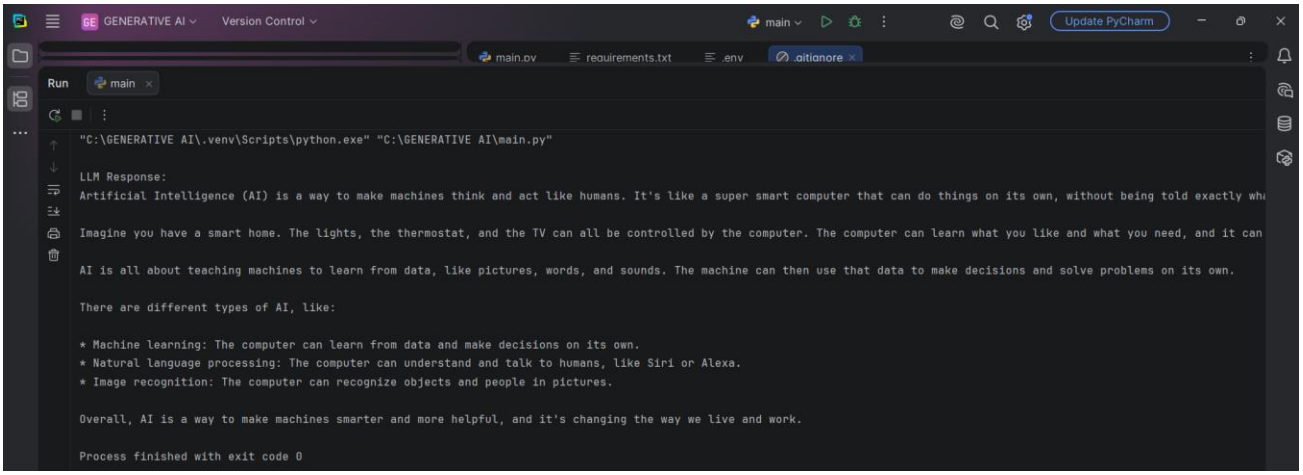
1. The Python application successfully loads the API key from the .env file.
2. The OpenAI client uses the API key for authentication.
3. The user can enter a prompt to ask a question.
4. The prompt is sent to the selected LLM through the OpenAI API.
5. The LLM processes the prompt and generates a response.
6. The generated response is displayed in the terminal.
7. Different prompts produce different AI-generated responses.
8. Common errors such as missing API keys, authentication errors and quota errors can be identified and handled.

9. Result

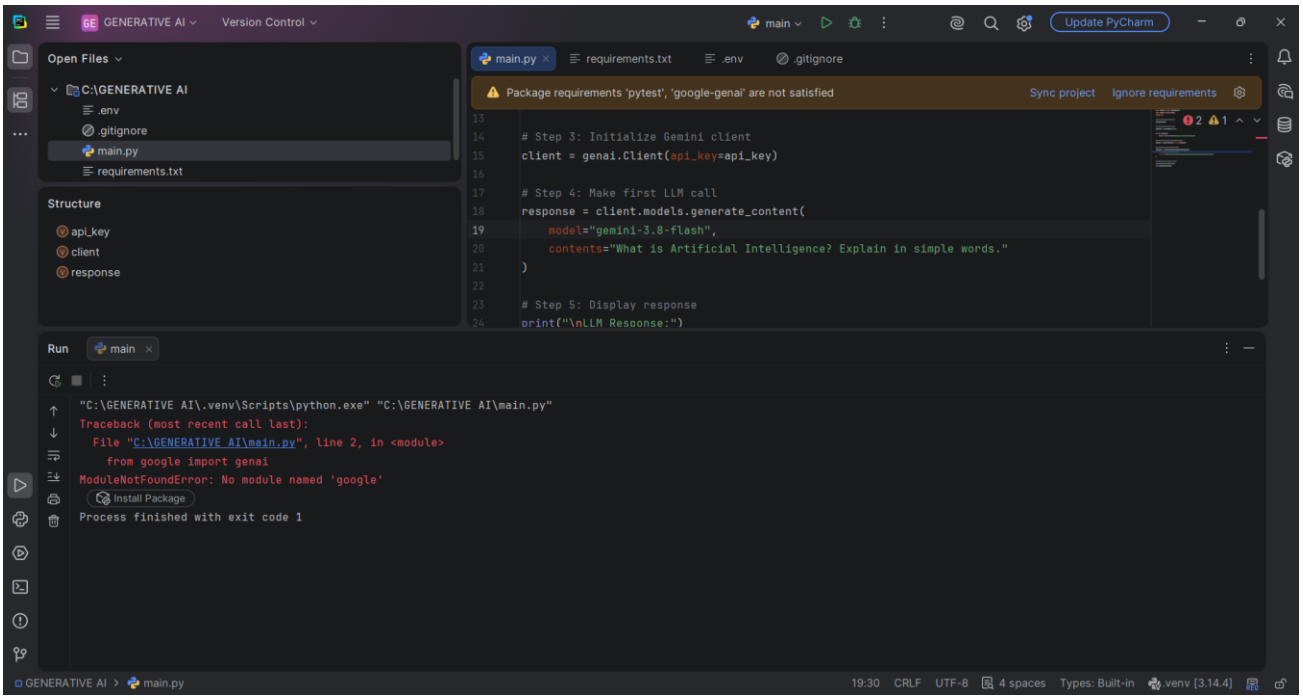
Successfully learned how to connect a Python application to a Large Language Model using the OpenAI API. The application uses an API key stored in a .env file, sends a user prompt and displays the AI-generated response.

Output:

Using ollama:



Using Gemini API:



10.Conclusion

The experiment helped us understand how Python communicates with an LLM through an API, how API keys are used for authentication and how AI-generated responses are obtained using prompts.